Fruit Ninja 小遊戲簡報講稿（繁體中文）

快速使用說明

- 受眾：非 IT 觀眾與學生。
- 語氣：簡單、溫暖、清楚。
- 節奏：每一頁約 45 到 75 秒。
- 總時長：約 8 到 10 分鐘。

開場（第 1 頁前）

大家好。 今天我想介紹我們的 Fruit Ninja 小遊戲專案。 這個專案最特別的地方是：我們不只可以用滑鼠，還可以用攝影機裡的手勢來玩。 我會用很容易懂的方式，講清楚背後的技術。

第 1 頁 - 你的手指就是刀

這一頁的核心很簡單。 你的手指，就是遊戲裡的刀。

你可以把這個遊戲想成一面「聰明的鏡子」。 攝影機會看見你的手。 程式會找到你的食指。 然後把刀畫在你手指移動的位置上。

所以看起來像魔法，其實是很清楚的流程。 輸入來自攝影機，然後遊戲會非常快地更新，大約每秒 60 次。

過渡句： 接下來我們看一下，攝影機畫面是怎麼一步步變成刀的位置。

第 2 頁 - 攝影機怎麼控制遊戲

這一頁可以用接力賽來理解。 每一步都會把資訊傳給下一步。

第 1 步：攝影機提供即時畫面。 第 2 步：手部辨識找到指尖。 第 3 步：平滑處理去掉小抖動。 第 4 步：座標映射把攝影機位置換成遊戲位置。 第 5 步：刀跟著映射後的點移動。

用簡單的話說，這就是刀為什麼看起來穩，不會一直抖。 我們把攝影機裡的雜訊動作，變成平滑動作。

過渡句： 接下來是關鍵規則：速度。

第 3 頁 - 漫畫版：一次揮動如何被判定

這一頁用「漫畫三格」來講判定邏輯。

第 1 格：遊戲先記錄你的揮動軌跡。 第 2 格：遊戲檢查兩個條件。 第一個條件：軌跡有沒有經過水果附近？ 第二個條件：速度夠不夠快？ 第 3 格：最後給出結果，切到或沒切到。

所以一次揮動要「位置對 + 速度夠」才算切中。 這就是為什麼隨便晃手，不一定會切到水果。

技術真相（簡單說）： 程式裡有一個最低切割速度規則。

過渡句： 接下來我們看「命中與未命中」是怎麼用幾何來判斷的。

第 4 頁 - 遊戲如何判斷命中或未命中

這裡我們用圖形比喻。 想像你用粗麥克筆畫了一條線。 這條粗線就是刀的軌跡。 水果會被看成一個圓。

如果粗線碰到圓，就是命中。 如果沒有碰到，就是未命中。

所以遊戲不是在猜。 它每一幀都在做幾何距離判斷。

過渡句： 接下來我們把鏡頭拉遠，看完整的遊戲迴圈。

第 5 頁 - 重複執行的遊戲迴圈

這是遊戲引擎的核心「配方」。 遊戲會不斷重複這幾個步驟。

1. 監聽輸入事件。
2. 讀取手或滑鼠位置。
3. 移動水果與物件。
4. 檢查碰撞。
5. 繪製畫面。
6. 播放聲音並顯示這一幀。

這個迴圈大約每秒執行 60 次。 所以操作才會感覺很跟手。 如果這個迴圈變慢，遊戲就會有卡頓感。

過渡句： 接下來比較一下兩種遊戲模式。

第 6 頁 - 經典模式 vs 生存模式

我們有兩種挑戰風格。

經典模式更適合練習。 你有 3 條命。 碰到炸彈或漏掉水果會扣 1 條命。

生存模式壓力更大。 你只有 1 條命。 一次重大失誤就會結束。

同一個引擎，同樣的揮動技巧，不同的難度體驗。

過渡句： 下一頁我會用「團隊分工」來解釋專案結構。

第 7 頁 - 每個 Python 檔案各司其職

我們的程式結構像學校活動小組。 每個檔案都有明確職責。

main.py 是隊長，負責主迴圈與流程。 input_manager.py 是翻譯員，把手勢或滑鼠轉成遊戲輸入。 sensors.py 負責攝影機與手部追蹤。 physics.py 負責命中判定。 game_engine.py 負責分數與生命規則。 game_objects.py 負責水果、刀、炸彈與特效物件。 ui_manager.py 負責選單介面。 audio_manager.py 負責音樂與音效。

這樣的結構讓專案更容易開發、解釋與除錯。

過渡句： 最後我們用一個「迷思小複盤」收尾。

第 8 頁 - 複盤與迷思澄清

迷思一：遊戲只看一張照片。 真相：它看的是連續影片幀。

迷思二：只要動就一定能切到。 真相：必須同時滿足位置與速度條件。

迷思三：命中判定是隨機的。 真相：判定依據是幾何規則。

迷思四：所有邏輯都寫在一個檔案。 真相：專案是模組化分工，像團隊合作。

結束句： 我們的專案把電腦視覺、遊戲設計與清楚的 Python 結構結合在一起，而且初學者也能理解。 謝謝大家。

可選問答提示

如果有人問：手勢為什麼會比較穩定？ 你可以回答：我們先做平滑處理，再把動作送進遊戲。

如果有人問：為什麼有些揮動沒切到？ 你可以回答：可能是太慢，或是軌跡沒有靠近水果。

如果有人問：為什麼要做兩種模式？ 你可以回答：經典模式適合練習，生存模式提供更高挑戰。